



OTUS

Онлайн образование

otus.ru



Проверить, идет ли запись

**Меня хорошо видно
&& слышно?**



Тема вебинара

Введение в планы запросов

Курс MS SQL Server



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в telegram



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом

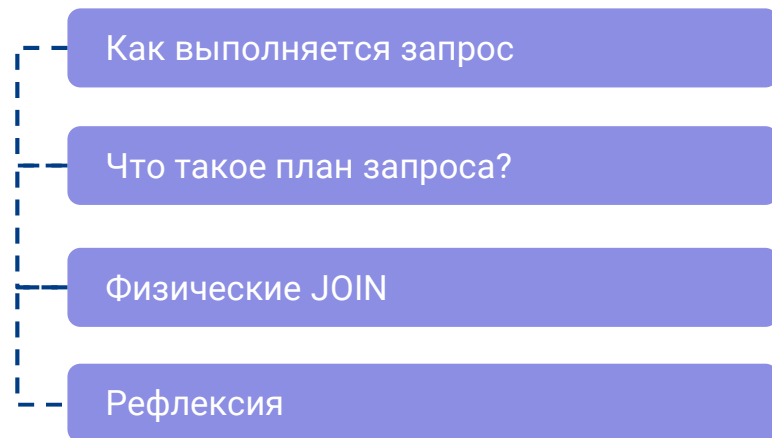


Документ



Ответьте себе или
задайте вопрос

Маршрут вебинара



Цели вебинара

После занятия вы сможете

1. Читать и объяснять план запроса
 2. Понимать какие индексы используются
 3. Оценивать ресурсоемкость разных запросов
-

Смысл

Зачем вам это уметь?

1. Чтобы оптимизировать запросы
-

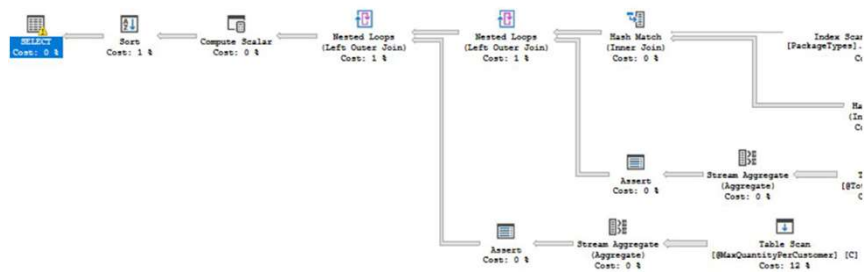


План выполнения запроса

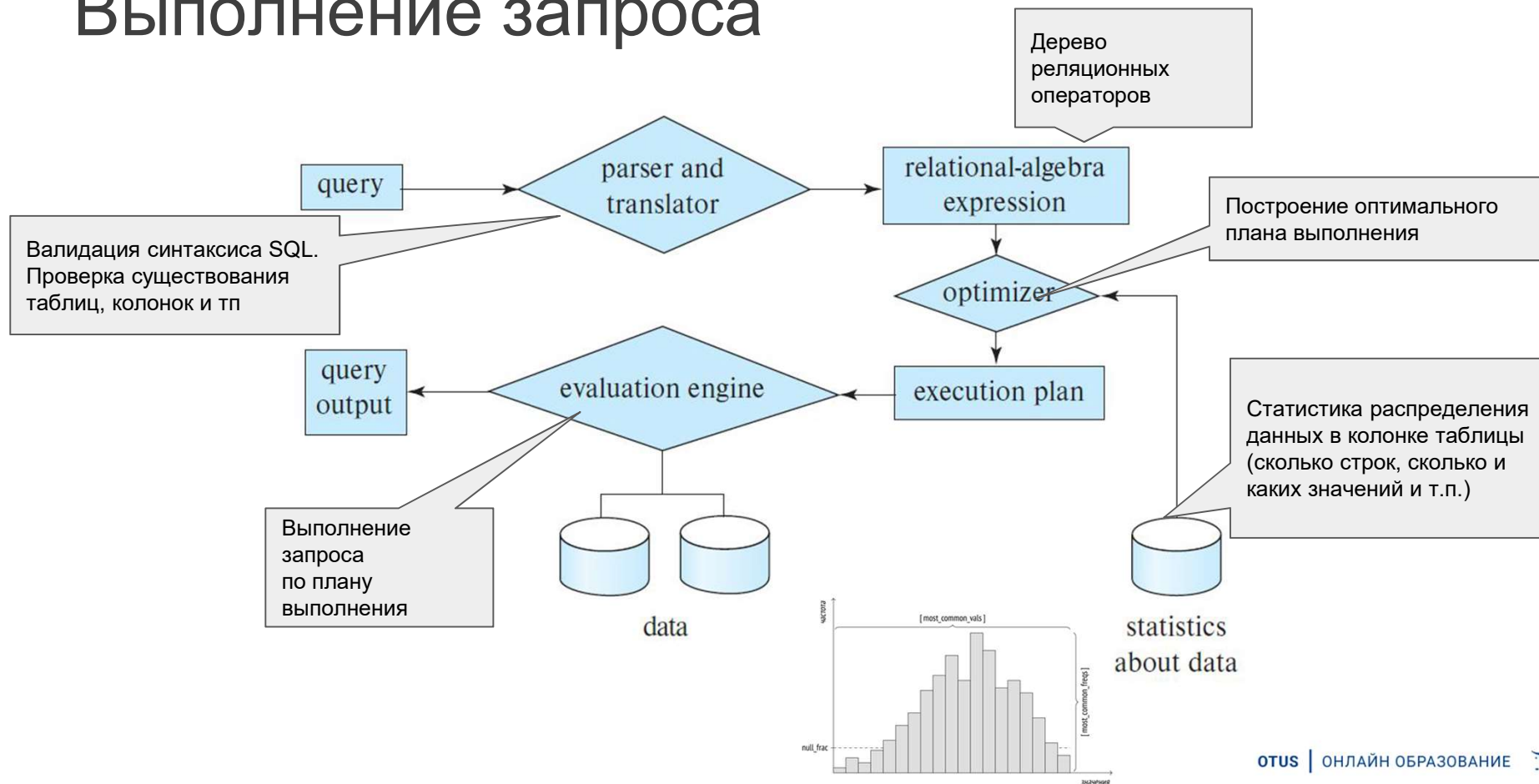


План запроса

- SQL — декларативный язык
 - Язык, в котором не задается пошаговый алгоритм решения задачи ("как" решить задачу), а некоторым образом описывается, "что" требуется получить в качестве результата.
 - Хотя есть и императивные расширения: IF, WHILE, LOOP и др. Например: "Купи хлеб".
- План выполнения — как получить результат "физически"
 - Например: "Выйди на улицу, пройди 5 шагов вперед, поверни направо, ... зайди в магазин, дай деньги, скажи ...".



Выполнение запроса



Особенности плана запроса

- Стоимостной оптимизатор (cost-based optimizer)
Бывает еще rule-based optimizer (на основе правил)
- SQL Server генерирует много планов
- Выбирается один план с наименьшей стоимостью |
(CPU, IO, память)

Сервер использует не самый лучший план, а
самый **лучший план, который смог найти** за определенное время.

Виды планов

Оценивается:

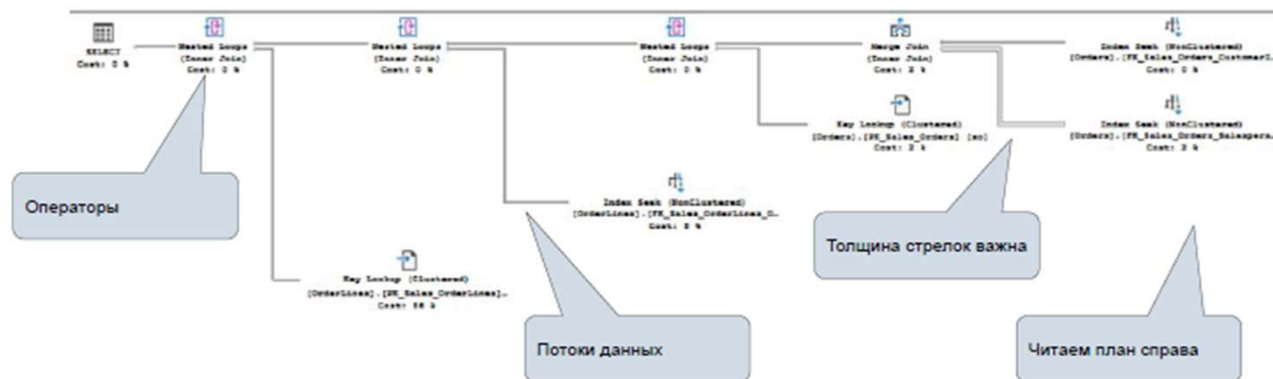
- Количество строк
- Необходимая память



Зачем читать план запроса?

План запроса – это то как сервер будет выбирать данные физически, план действий.

1. Какой индекс используется
2. В каком порядке делается JOIN
3. Что выбирается из буфера памяти
4. Сколько сервер тратит ресурса на операцию
5. Разницу между предполагаемым и действительным планом



Demo



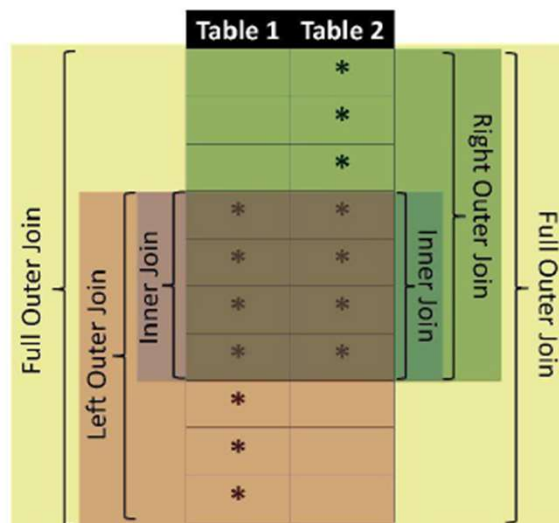
JOIN



Типы JOIN

Логические:

INNER, LEFT, RIGHT, FULL



Физические:



Nested Loops



Merge Join



Hash Match

InvoiceId	CustomerId	Total
1	2	500
2	3	1000
3	1	15000
4	2	100
5	3	1000
6	2	1020

CustomerId	CustomerName
1	Пупкин
2	Иванов
3	Петров

Nested loop

Вложенные циклы

Проходит по набору данных из таблицы A, по каждому значению в таблице A ищет соответствие в таблице B (если есть индекс, то использует его для поиска значения).



Nested Loops

Так далее по следующему значению таблицы A.

Хорош для небольшого набора данных (одна из таблиц должна быть небольшой, она и будет выбрана для внешнего цикла)

```
Для каждой строки [r] из [Ведущая таблица]
  Для каждой строки [s] из [Ведомая таблица]
    Если УдовлетворяетУсловию ([r],[s],[Условие соединения])
      Вывести ([r],[s]);
```



Merge join

Слияние

Используется, когда оба набора отсортированы (имеют индекс)

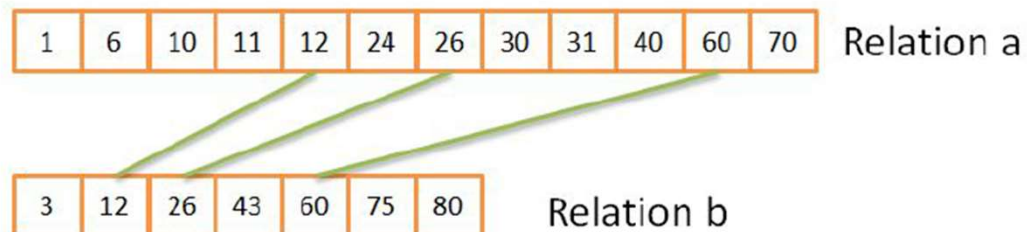
Может использовать tempdb если в первом наборе много дубликатов

Лучший вариант для больших наборов данных



Merge Join

Merge Join



Хеш-функция

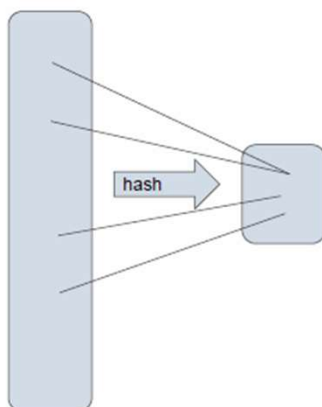
Хеш-функция (англ. *hash function* от *hash* — «превращать в фарш», «мешанина»), или **функция свёртки** — функция, осуществляющая преобразование массива входных данных произвольной длины в выходную битовую строку установленной длины, выполняемое определённым алгоритмом.

[Хеш-функция \(wikipedia\)](#)

Хеш-функция может вычислять «хеш» как остаток от деления входных данных на M :

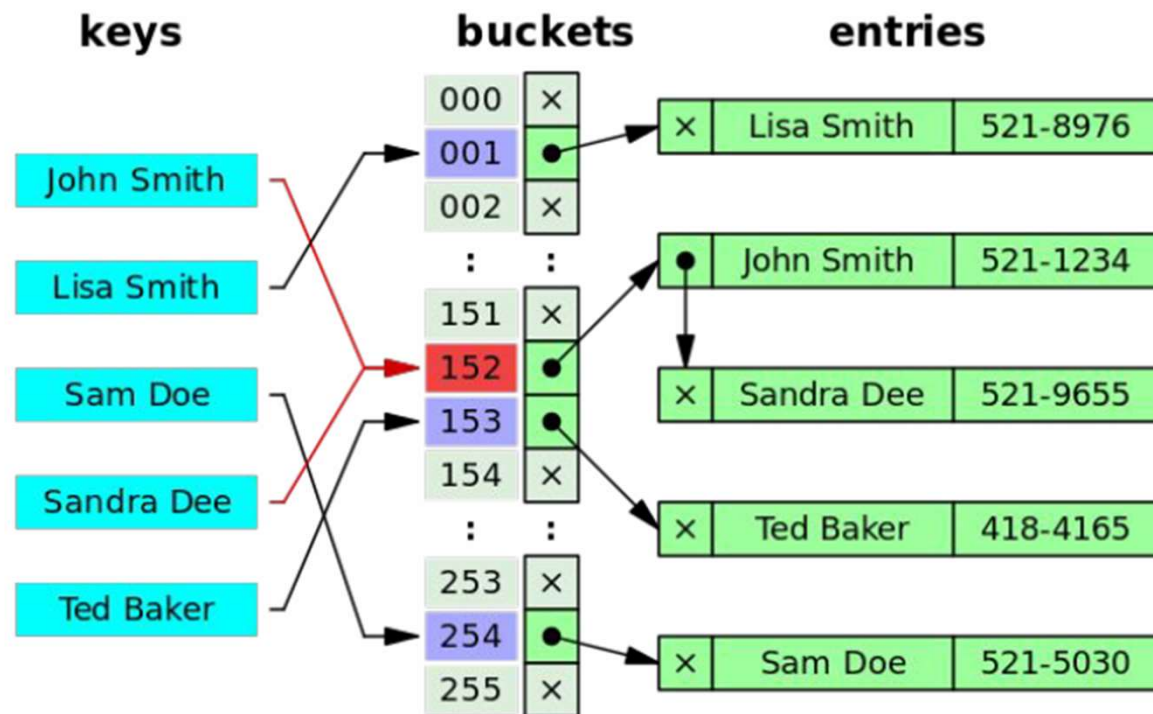
$$h(k) = k \mod M,$$

где M — количество всех возможных «хешей» (выходных данных).



Ввод		Вывод
Fox	криптографическая хэш-функция	DFCD 3454 BBEA 788A 751A 696C 24D9 7009 CA99 2D17
The red fox jumps over the blue dog	криптографическая хэш-функция	0086 462B F57D CB22 823C ACC7 6CD1 90B1 EE6E 3AEC
The red fox jumps over the blue dog	криптографическая хэш-функция	8FD8 7558 7851 4F32 D1C6 76B1 79A9 0DA4 AEF2 4819
The red fox jumps over the blue dog	криптографическая хэш-функция	FCD3 7FDB 5AF2 C6FF 915F D401 C0A9 7D9A 46AF FB45
The red fox jumps over the blue dog	криптографическая хэш-функция	8ACA D682 D588 4C75 4EF4 1799 7D88 BCF8 92B9 6A6C

Хеш-таблица



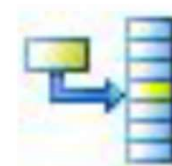
Hash Join

2 фазы:

Build (компоновка, построение, распределение)

строится хэш таблица по наименьшей таблице

- Для каждого значения в таблице1 считается хэш
- Сохраняется значение в хэш-таблицу, вычисленный хэш используется как ключ



Hash Match

Probe (проба)

- Для каждой строки из таблицы2 считается значение хэш по полям, которые указаны в join (оператор =)
- Ищется хэш в хэш-таблице, проверяются значения полей

Если build таблица не помещается в память, она будет помещена на диск в tempdb.

По сути строится хэш-индекс в памяти

Hash Join

CustomerId	CustomerName	Hash
1	Пупкин	
2	Иванов	
3	Петров	

InvoiceId	CustomerId	Total
1	2	500
2	3	1000
3	1	15000
4	2	100
5	3	1000
6	2	1020



Hash Join

Build Table

CustomerId	CustomerName	Hash
1	Пупкин	050c5d21
2	Иванов	
3	Петров	

InvoiceId	CustomerId	Total
1	2	500
2	3	1000
3	1	15000
4	2	100
5	3	1000
6	2	1020

Hash Join

Build Table

CustomerId	CustomerName	Hash
1	Пупкин	050c5d22
2	Иванов	051a5f21
3	Петров	052afd25

Probe Table

Hash	InvoiceId	CustomerId	Total
051a5f21	1	2	500
052afd25	2	3	1000
050c5d22	3	1	15000
	4	2	100
	5	3	1000
	6	2	1020

Result Table

CustomerId	CustomerName	InvoiceId	CustomerId	Total
2	Иванов	1	2	500
3	Петров	2	3	1000

```
[ХешТаблица] - СтроитьХешТаблицу([МеньшаяТаблица], [Имена колонок МеньшейТаблицы по которым будет делаться соединение]);  
Для каждой строки [r] из [БольшаяТаблица]  
  Вывести ([r], ИскатьВХешТаблице([ХешТаблица],[Имена колонок БольшойТаблицы по которым делается соединение]));
```


Операторы



Nested Loops

Nested loops

- Одна из таблиц небольшого размера



Merge Join

Merge Join

- Оба набора данных проиндексированы
- Хорош для больших наборов данных



Hash Match

Hash Match

- Неиндексированные наборы данных
- Обе таблицы большие
- Оператор соединения =
- Может использовать tempdb



Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет



Рефлексия



Вопросы для проверки

О чем мы говорили сегодня?

- Что такое план запроса?
- Что такое куча?
- Что такое кластерный индекс? Сколько их может быть на таблице?
- Чем estimated план отличается от actual?
- Что такое Hash Join? По каким таблицам строится хэш функция?

Рефлексия



С какими основными мыслями
и инсайтами уходите с вебинара?



Как будете применять на практике то,
что узнали на вебинаре?

**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**

